

Pliki i strumienie w języku Java

Zbigniew Marszalek

Pliki i strumienie

Podczas tworzenia aplikacji jesteśmy zmuszeni do zapisywania informacji na twardych nośnikach, tak aby można byłoby nimi manipulować podczas ponownego uruchomienia aplikacji. Informacje takie możemy zapisywać w bazach danych albo w plikach umieszczonych na zewnętrznych nośnikach. Przez plik rozumiemy ciąg bajtów zapisany na dysku twardym lub innym nośniku umożliwiającym jego odczytanie, zapisanie albo modyfikację. Istnieje tutaj jeszcze jedno pojęcie strumień, który jest kanałem łączącym dwa różne punkty podczas przesyłania informacji. W środowisku Java rozróżniamy następujące typy strumieni:

Typy strumieni

- Strumienie plików przeznaczone do obsługi danych zapisanych na trwałych nośnikach.
- Strumienie sieciowe zajmujące się transferem danych przez sieć.
- Strumienie pamięci umożliwiające dostęp do danych umieszczonych w pamięci komputera.

Za pomocą strumieni możemy wykonywać takie operacje jak: kompresja i dekompresja danych, ich serializacja i deserializacja oraz szyfrowanie i rozszyfrowanie danych. Pojęcia przedstawione podczas wykładu:

Tematy wykładu:

- Manipulowanie plikami i katalogami.
- Odczytywanie i zapisywanie informacji do pliku.
- Strumienie.
- Używanie klasy `BufferedStream` w celu poprawienia wydajności aplikacji.
- Zastosowanie klasy `FileStream` do odczytu i zapisu danych na dysku.

W języku Java zdefiniowana jest klasa `io`, która pozwala na synchroniczne i asynchroniczne odczytywanie i zapisywanie danych do pliku. Klasę dołączamy poleceniem:

```
import java.io.*;
```

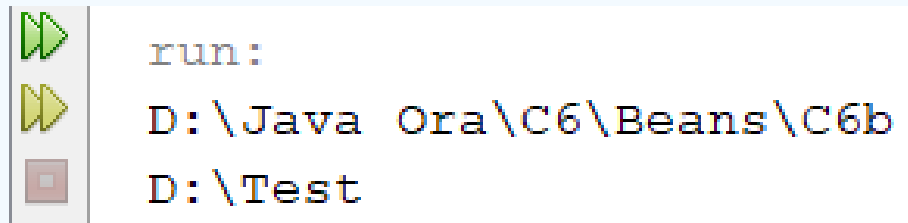
Praca z plikami – obecnie używany katalog

Możemy otrzymać ścieżkę obecnie używanej kartoteki przez naszą aplikację za pomocą polecenia:

```
String kat = System.getProperty("user.dir");
```

Zmianę obecnie używanej kartoteki możemy wykonać poleceniem *System.setProperty()*. Metoda ta przyjmuje kilka argumentów, które są ścieżkami naszej aplikacji. Jeżeli pierwszy argument nie zawiera poprawnej ścieżki dostępu metoda przechodzi do pobrania ścieżki z kolejnego argumentu.

```
System.setProperty("user.dir", "D:\\Test");
```



```
run:  
D:\\Java Ora\\C6\\Beans\\C6b  
D:\\Test
```

Sprawdzanie istnienia pliku

Możemy dokonać sprawdzenia pliku w żądanym katalogu za pomocą metody *exists()* z klasy *File* np.:

```
File danePlik = new File("dane.txt");  
// Utworzenie obiektu File  
Boolean daneIstnieje = danePlik.exists();  
// Sprawdzenie istnienia pliku.  
if(daneIstnieje)  
    System.out.println("Plik dane.txt istnieje.");  
else  
    System.out.println("Plik dane.txt nie istnieje.");
```



run:



Plik dane.txt nie istnieje.

Ścieżki dostępu do pliku

Oprócz ścieżki względnej do pliku istnieje ścieżka kanoniczna. Ścieżka bezwzględna identyfikuje plik jednoznacznie w systemie plików. Kanoniczna ścieżka jest najprostszą ścieżką, która unikatowo identyfikuje plik w systemie plików. Jedyną różnicą między obydwojema ścieżkami jest najprostsza postać

kanoniczna ścieżki. Przykładowo w systemie Windows, jeśli ścieżka bezwzględna do dane.txt jest C:\users\dane.txt, ścieżka C:\users\test\..

\dane.txt reprezentuje również ścieżki dla tego samego pliku. Dwóch następujących po sobie kropek (..) używa się do reprezentacji poziom wyżej w hierarchii plików. Wśród dwóch ścieżek druga postać nie jest najprostszą.

Za pomocą metod *getAbsolutePath()* i *getCanonicalPath()* można uzyskać postać bezwzględną i kanoniczną reprezentacji obiektu File. Należy zauważyć, że w programie Java należy użyć podwójnych odwróconych znaków ukośnika w literale ciągu znaków do reprezentowania pojedynczego znaku ukośnika np..

ścieżka C:\users\sharan musi być zapisany jako "C:\\users\\sharan".

Program poniżej przedstawia sposób uzyskać bezwzględną i kanoniczną ścieżki pliku.

```
import java.io.*;
public class C6b2 {
    public static void printFilePath(String pathname) {
        File f = new File(pathname);
        System.out.println("Bazwa pliku: " + f.getName());
        System.out.println("Plik istnieje: " + f.exists());
        System.out.println("Ścieżka bezwzględna: " +
            f.getAbsolutePath());
        try {
            System.out.println("Ścieżka kanoniczna: " +
                f.getCanonicalPath());
        }
        catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

```
public static void main(String[] args) {  
    String workingDir =  
        System.getProperty("user.dir");  
    System.out.println  
        ("Katlg roboczy: " + workingDir);  
    System.out.println  
        ("-----");  
    printFilePath("dane.txt");  
    System.out.println  
        ("-----");  
    printFilePath(".. " + File.separator  
        + "notatki.txt");  
}  
}
```

Ścieżka bezwzględna i kanoniczna

run:

Katlg roboczy: D:\Java Ora\C6\Beans\C6b2

Bazwa pliku: dane.txt

Plik istnieje: false

Ścieżka bezwzględna: D:\Java Ora\C6\Beans\C6b2\dane.txt

Ścieżka kanoniczna: D:\Java Ora\C6\Beans\C6b2\dane.txt

Bazwa pliku: notatki.txt

Plik istnieje: false

Ścieżka bezwzględna: D:\Java Ora\C6\Beans\C6b2\..\notatki.txt

Ścieżka kanoniczna: D:\Java Ora\C6\Beans\notatki.txt

Tworzenie i usuwanie plików

Metoda *CreateNewFile()* tworzy nowy, pusty plik, jeśli katalog o określonej nazwie już istnieje.

```
File danyPlik = new File("dane.txt");  
// Tworzenie obiektu zbioru za pomocą  
// ścieżki absolutnej  
boolean plik = danyPlik.createNewFile();  
// Tworzenie zbioru
```

Zwraca wartość *true*, jeśli plik jest tworzony pomyślnie. W przeciwnym razie zwraca *false*. Metoda wygeneruje *IOException* jeśli wystąpi błąd we/wy. Można również utworzyć tymczasowy plik w katalogu domyślnym za pomocą metody *createTempFile()*.

```
File temp = File.createTempFile("kkk", ".txt");
```

Metody *mkdir()* lub *makedirs()* służą do tworzenia nowego katalogu.

Metoda *mkdir()* Metoda tworzy katalog, tylko wtedy, gdy katalogi określone przez parametr *nazwa_ścieżki* już istnieją. Przykładowo, jeśli chcemy utworzyć nowy katalog o nazwie *home* w katalogu *users* na dysku C, po systemem Windows, konstruujemy obiekt pliku reprezentujący nazwę ścieżki w następujący sposób:

```
File nowaKartoteka = new File("C:\\users\\home");
```

. Usuwanie pliku jest łatwe. Należy użyć metodę *delete()* klasy *File*, aby usunąć plik lub katalog. Katalog musi być pusty, zanim można go usunąć. Metoda zwraca *true*, jeśli plik lub katalog jest usunięty. W przeciwnym razie zwraca *false*. Można również opóźnić usunięcie pliku do zakończenia działania JVM za pomocą metody *deleteOnExit()*. Jest to przydatne, jeśli tworzymy pliki tymczasowe w programie, który chcemy usunąć, gdy zamykamy program.

Usuwanie pliku

```
File danyPlik1 = new File("dane1.txt");  
danyPlik1.delete();  
// Natychmiastowe usunięcie pliku  
danyPlik1 = new File("dane1.txt");  
danyPlik1.deleteOnExit();  
// Usunięcie pliku, kiedy zostaje  
// zatrzymana maszyna wirtualna Jawy (JVM)
```

Zmiana nazwy pliku – metoda *renameTo()*

```
boolean plikZmieniony =  
    staryPlik.renameTo(nowyPlik);  
if (plikZmieniony)  
    System.out.println(staryPlik +  
        " zmieniony na " + nowyPlik);  
else  
    System.out.println("Zmiana " + staryPlik +  
        " na " + nowyPlik + " się niepowiodła");
```



run:



Zmiana stare_dane.txt na nowe_dane.txt się niepowiodła

```
package c6b4;
import java.io.*;
public class C6b4 {
    public static void printFileDetails(File f) {
        System.out.println("Absolutna ścieżka: " +
            f.getAbsolutePath());
        System.out.println("Plik istnieje: " + f.exists());
        System.out.println("-----");
    }
    public static void main(String[] args) {
        try {
            File newFile = new File("nowy_plik.txt");
            System.out.println("Przed utworzeniem pliku:");
            printFileDetails(newFile);
        }
    }
}
```

```
        boolean fileCreated = newFile.createNewFile();  
// Utworzenie nowego pliku  
        if (!fileCreated) {  
            System.out.println  
                (newFile + " nie może być utworzony.");  
        }  
        System.out.println("Po utworzeniu nowego pliku:");  
        printFileDetails(newFile);  
  
// Skasowanie nowego pliku  
        newFile.delete();  
  
        System.out.println("Po usunięciu nowego pliku:");  
        printFileDetails(newFile);
```

```
// Ponowne utworzenie nowego pliku
newFile.createNewFile();

System.out.println
    ("Po ponownym utworzeniu nowego pliku:");
printFileDetails(newFile);

// Powiedzenie JVM aby skasował plik po zakończeniu
newFile.deleteOnExit();

System.out.println
    ("Po użyciu metody deleteOnExit():");
printFileDetails(newFile);
```

```
// Utworzenie nowego pliku i zmiana jego nazwy
File pierwszyPlik = new File("pierwszy_plik.txt");
File drugiPlik = new File("drugi_plik.txt");

fileCreated = pierwszyPlik.createNewFile();
if (fileCreated || drugiPlik.exists()) {
    System.out.println("Przed zmianą nazwy:");
    printFileDetails(pierwszyPlik);
    printFileDetails(drugiPlik);

    boolean renamedFlag =
        pierwszyPlik.renameTo(drugiPlik);
}
```

```
        if (!renamedFlag) {
            System.out.println("Nie mogę zmienić " +
                               pierwszyPlik);
            System.out.println
                ("Po zmianie nazwy pliku:");
            printFileDetails(pierwszyPlik);
            printFileDetails(drugiPlik);
        }
    }
}
catch (IOException e) {
    e.printStackTrace();
}
}
```



run:



Przed utworzeniem pliku:



Absolutna ścieżka: D:\Java Ora\C6\Beans\C6b4\nowy_plik.txt



Plik istnieje: false

Po utworzeniu nowego pliku:

Absolutna ścieżka: D:\Java Ora\C6\Beans\C6b4\nowy_plik.txt

Plik istnieje: true

Po usunięciu nowego pliku:

Absolutna ścieżka: D:\Java Ora\C6\Beans\C6b4\nowy_plik.txt

Plik istnieje: false

Po ponownym utworzeniu nowego pliku:

Absolutna ścieżka: D:\Java Ora\C6\Beans\C6b4\nowy_plik.txt

Plik istnieje: true

```
Po użyciu metody deleteOnExit():
Absolutna ścieżka: D:\Java Ora\C6\Beans\C6b4\nowy_plik.txt
Plik istnieje: true
-----

Przed zmianą nazwy:
Absolutna ścieżka: D:\Java Ora\C6\Beans\C6b4\pierwszy_plik.txt
Plik istnieje: true
-----

Absolutna ścieżka: D:\Java Ora\C6\Beans\C6b4\drugi_plik.txt
Plik istnieje: true
-----

Nie mogę zmienić pierwszy_plik.txt
Po zmianie nazwy pliku:
Absolutna ścieżka: D:\Java Ora\C6\Beans\C6b4\pierwszy_plik.txt
Plik istnieje: true
-----

Absolutna ścieżka: D:\Java Ora\C6\Beans\C6b4\drugi_plik.txt
Plik istnieje: true
-----
```

Kopiowanie plików

Klasa `File` nie dostarcza metod do kopiowania plików. Aby skopiować plik, należy utworzyć nowy plik, odczytywać zawartość oryginalnego pliku i wgrać do nowego pliku. Dopiero nowe wersje języka Java z interfejsem NIO 2.0 API zapewniają bezpośredni sposób kopiowania zawartość pliku i jego atrybutów.

Rozmiar pliku w bajtach można uzyskać za pomocą metody *length()* klasy `File`.

```
File myFile = new File("myfile.txt");  
long fileLength = myFile.length();
```

Jeśli obiekt `File` reprezentujący plik nie istnieje, metoda *length()* zwraca zero. Jeśli jest to nazwa katalogu, wartość zwracana nie jest określona. Należy pamiętać, że zwracany typ metody *length()* jest `long`, nie `int`.

Wyświetlanie listy wszystkich plików i katalogów

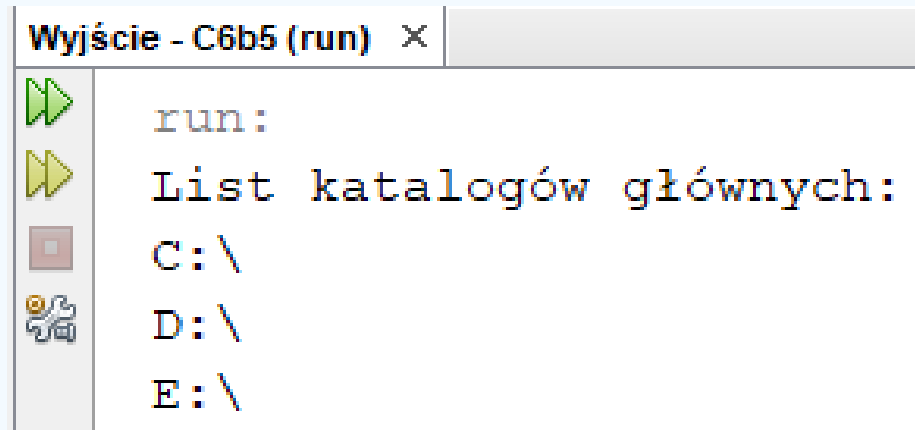
Listę katalogów głównych dostępnych w systemie plików można uzyskać plików za pomocą metody statycznej *listRoots()* klasy *File*. Zwraca tablicę obiektów *File*.

```
File[] roots = File.listRoots();  
//Pobieranie listy katalogów głównych
```

Sposób zapisów katalogów głównych różni się na różnych platformach. W systemie Windows mamy katalog główny dla każdego dysku (np. C:\, A:\, D:\, itp.). W systemie UNIX mamy jeden katalog główny katalog reprezentowany przez ukośnik. Poniżej został przedstawiony program, w którym otrzymywane wynik zależą od systemu operacyjnego.

Wyświetlenie listy katalogów głównych

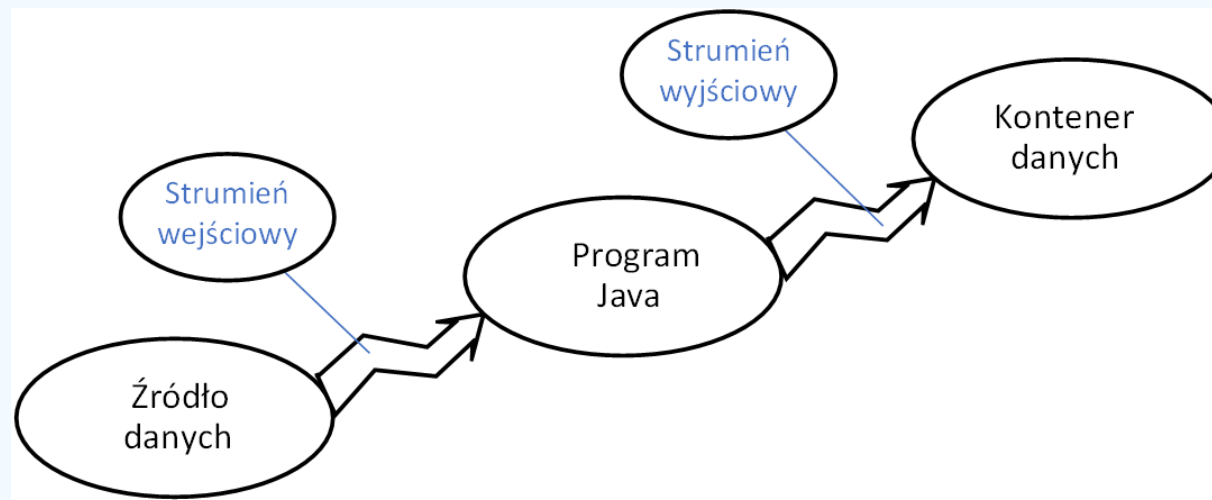
```
File[] roots = File.listRoots();  
System.out.println("List katalogów głównych:");  
for(File f : roots)  
    System.out.println(f.getPath());
```



```
Wyjście - C6b5 (run) ×  
run:  
List katalogów głównych:  
C:\  
D:\  
E:\
```

Strumienie wejścia/wyjścia

Strumienie wejścia/wyjścia w dosłownym rozumieniu słowo strumienia jest "nieprzerwany przepływ czegoś." W Java we/wy strumień oznacza nieprzerwany przepływ (lub sekwencyjny przepływ) danych. Dane w strumieniu mogą być bajtami, znakami, obiektami, itp. Strumień, który łączy program z systemem zawierającym dane nazywa się strumieniem wyjściowym. Odwrotnie jeśli dane przepływają z programu do pliku zapisanego w systemie mamy zdefiniowany strumień wyjściowy. Innymi słowy program w języku Java odczytuje dane ze strumienia wejściowego i zapisuje dane do strumienia wyjściowego.



Czytanie z pliku za pomocą strumienia wejściowego

Możemy utworzyć plik dane.txt z tekstu i zapisz go w bieżącym katalogu roboczym. Następujące kroki są potrzebne do odczytu z pliku:

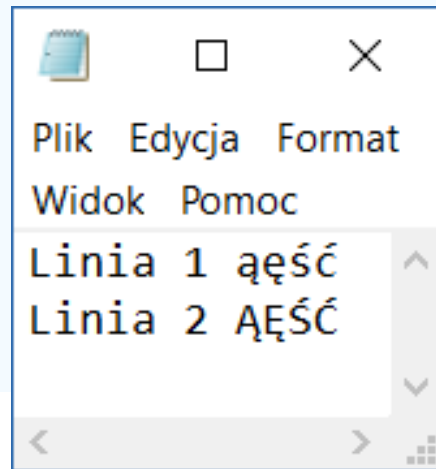
- identyfikacji źródła danych, który w tym przypadku jest ścieżka do pliku.
- utworzyć strumień wejściowy przy użyciu pliku
- odczytać dane z pliku, przy użyciu strumienia wejściowego
- Zamknąć strumień wejściowy

```
String srcFile = "dane.txt";  
    // Źródło danych  
    try(FileInputStream fin =  
        new FileInputStream(srcFile)) {  
// Utworzenie strumienia i połączenie z plikiem  
        byte byteData;  
        while ((byteData = (byte)fin.read()) != -1) {  
            System.out.print((char)byteData);  
        }  
        fin.close();  
// Zamknięcie strumienia  
    }
```

```

catch (FileNotFoundException e) {
    System.out.println("Błąd!" +
        e.getLocalizedMessage());
}
catch (IOException e) {
    System.out.println("Błąd!" +
        e.getLocalizedMessage());
}

```



```

run:
Linia 1 0↑77
Linia 2 0070

```

```

run:
Błąd! dane.txt (Nie można odnaleźć określonego pliku)

```

Dekodowanie polskich znaków – windows1250

```
import java.io.*;  
import java.nio.charset.Charset;  
import java.nio.*;
```

	run:
	Linia 1 aęść
	Linia 2 AĘŚĆ

```
byte byteData;  
while ((byteData = (byte)fin.read()) != -1) {  
    Charset cset = Charset.forName("Cp1250");  
    byte[] b = new byte[1];  
    b[0] = byteData;  
    ByteBuffer buffer = ByteBuffer.wrap(b);  
    CharBuffer c = cset.decode(buffer);  
    System.out.print(c.toString());  
}
```

Zbiory znaków

Obiekt klasy *Charset* uzyskujemy, wywołując metodę statyczną *forName*, której podajemy oficjalną nazwę zbioru znaków lub jeden z jej synonimów:

```
Charset cset =  
    Charset.forName("windows-1250");
```

Dysponując zbiorem znaków, możemy użyć go do konwersji łańcuchów Unicode do sekwencji bajtów np..

```
String str = ...;  
ByteBuffer buf = cset.encode(str);  
byte[] bytes = buf.array();
```

Natomiast aby dokonać konwersji w kierunku przeciwnym, potrzebny będzie bufor. Można wykorzystać metodę statyczną *wrap* tablicy *ByteBuffer*, aby przekształcić tablicę bajtów w *buffer*. W wyniku działania metody *decode* otrzymamy obiekt klasy *CharBuffer*. Wystarczy wywołać metodę *toString*, aby uzyskać łańcuch znaków.

```
byte[] bytes = ...;  
ByteBuffer bbuf =  
    ByteBuffer.wrap(bytes, offset, length);  
CharBuffer cbuf = cset.decode(bbuf);  
String str = cbuf.toString();
```

Zapisywanie danych do pliku

Zapis danych do pliku odbywa się za pomocą strumienia wyjściowego. Klasa *FileOutputStream* posiada przeciążoną metodę *write()* do zapisu jednego lub ciągu bajtów do pliku. Należy umieścić kod zapisujący dane do strumienia wyjściowego w bloku *try-catch*, ponieważ metoda może wyrzucić wyjątek *IOException*. Jeśli chcemy zapisać obiekt typu *String* np. "Hello" do strumienia wyjściowego, należy przekonwertować łańcuch na ciąg bajtów. Klasa *String* posiada metodę *getBytes()*, która zwraca tablicę bajtów reprezentujący ciąg. Możemy to zapisać w następujący sposób:

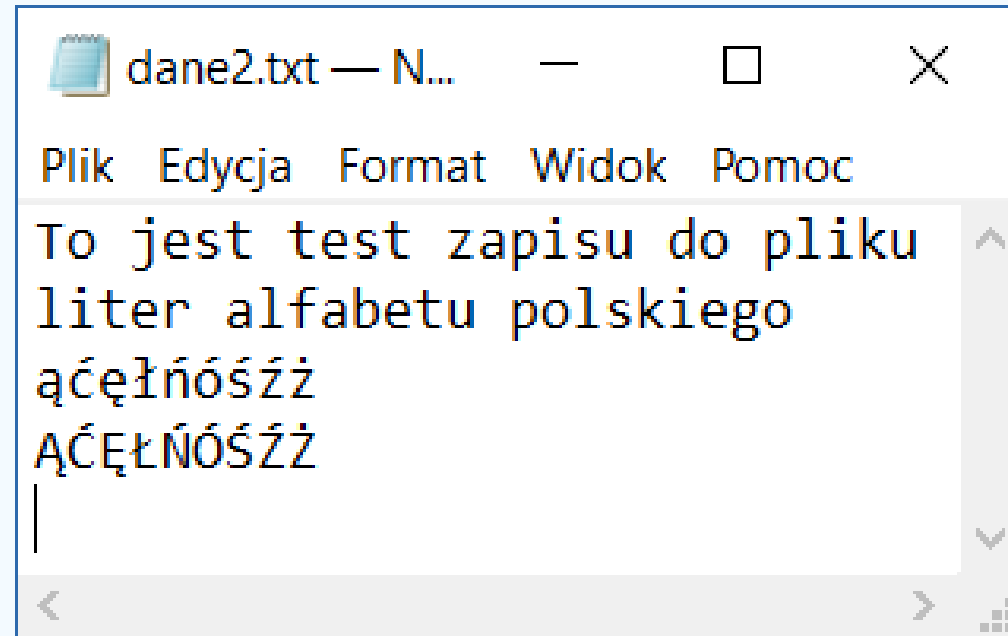
```
String text = "Hello";  
byte[] textBytes = text.getBytes();  
fos.write(textBytes);
```

```
package c6c;
import java.io.*;
public class C6c {
    public static void main(String[] args) {
        String wP = "dane2.txt";
        String sL =
            System.getProperty("line.separator");
        // Pobranie separatora linii dla platformy
        String line1 =
            "To jest test zapisu do pliku";
        String line2 =
            "liter alfabetu polskiego" ;
        String line3 = "ąćęłńóśźż";
        String line4 = "ĄĆĘŁŃÓŚŻŻ";
    }
}
```

```
try (FileOutputStream fos =  
    new FileOutputStream(wP)) {  
    fos.write(line1.getBytes());  
    fos.write(sL.getBytes());  
    fos.write(line2.getBytes());  
    fos.write(sL.getBytes());  
    fos.write(line3.getBytes());  
    fos.write(sL.getBytes());  
    fos.write(line4.getBytes());  
    fos.write(sL.getBytes());  
    fos.flush();  
// Opróżnienie bufora strumienia
```

```
        System.out.println  
            ("Text has been written to " +  
             (new File(wP)).getAbsolutePath());  
// Wyświetlenie ścieżki zapisanego pliku  
    }  
    catch (FileNotFoundException e) {  
        System.out.println("Błąd!" +  
                             e.getLocalizedMessage());  
    }  
    catch (IOException e) {  
        System.out.println("Błąd!" +  
                             e.getLocalizedMessage());  
    }  
}
```

Otrzymany plik



run:



Text has been written to D:\Java Ora\C6\Beans\C6c1\dane2.txt



BUILD SUCCESSFUL (total time: 0 seconds)

Zapisywanie polskich znaków – windows1250

Jak widzieliśmy w poprzednim programie litery polskich znaków zostały zapisane poprawnie do pliku. Nastąpiło to na skutek domyślnych ustawień programu NetBeans IDE. Gdybyśmy jednak pracowali w konsoli systemu musielibyśmy sami dokonać konwersji ze znaków zapisanych w Unicode do windows1250 np..

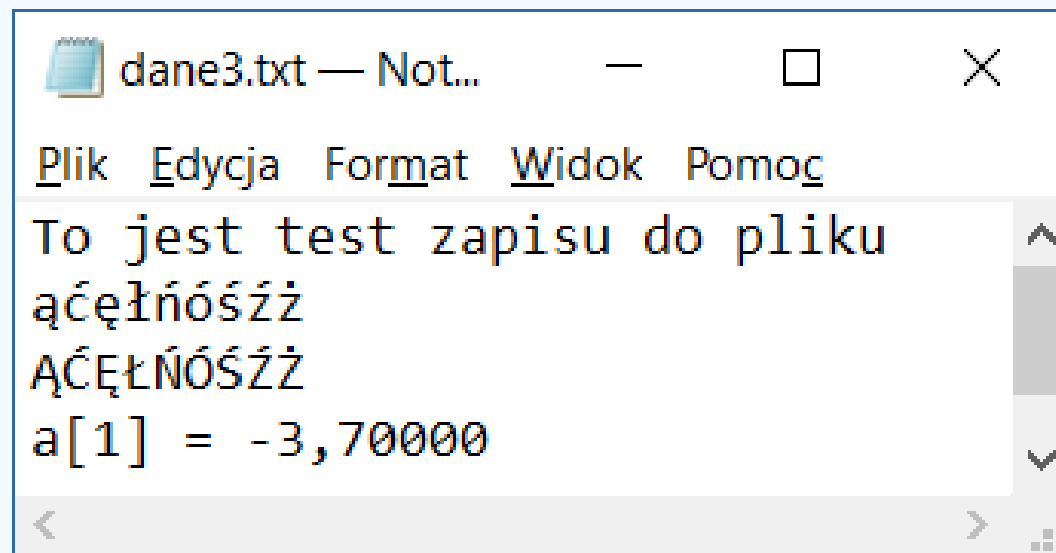
```
Charset cs = Charset.forName("Cp1250");  
// Kodowanie znaków polski.  
ByteBuffer bufor = cs.encode(line1);  
byte[] bayty = bufor.array();  
fos.write(bayty);  
fos.write(sL.getBytes());
```

Klasa *PrintStream*

Klasa *PrintStream* udostępnia wszystkie możliwości w zakresie obsługi danych wyjściowych, które są znane z klasy *System*. Ta cecha czyni z *Print Stream* jedną z najczęściej wykorzystywanych klas Javy. Klasa *PrintStream* obsługuje metody *print()*, *println()* i *printf()* dla wszystkich typów danych, włącznie z obiektami klasy *Object*. Jeśli argument któregoś z tych metod nie jest typem prostym, to klasa *PrintStream* automatycznie wywołuje metodę *toString()* i wyświetla uzyskany w ten sposób wynik. Metoda *printf()* umożliwia precyzyjne określenie zapisywanych danych. Metoda *printf()* wykorzystuje do zapisu danych klasę *Formatter*. Stosowanie tej metody nie odbiega od znanej z języka C i C++ funkcji *printf()*.

```
String dP = "dane3.txt";
try (PrintStream ps = new PrintStream(dP)) {
    ps.println("To jest test zapisu do pliku");
    ps.println("ąćęłńóśźż");
    ps.println("ĄĆĘŁŃÓŚŹŻ");
    ps.printf("a[%d] = %g%n", 1, -3.7);
// Zapis do pliku danych
    ps.flush();
    System.out.println("Text has been written to " +
        (new File(dP).getAbsolutePath()));
}
catch (FileNotFoundException e) {
    System.out.println("Błąd!" + e.getLocalizedMessage());
}
```

Otrzymane wyniki



```
dane3.txt — Not...  
Plik Edycja Format Widok Pomoc  
To jest test zapisu do pliku  
ąćęłńóśźź  
ĄĆĘŁŃÓŚŹŹ  
a[1] = -3,70000
```



run:



Text has been written to D:\Java Ora\C6\Beans\C6d\dane3.txt



BUILD SUCCESSFUL (total time: 0 seconds)



Czytanie i zapisywanie prymitywnych typów danych

Obiekt klasy *DataInputStream* jest używany do odczytu wartości typów danych w sposób niezależny od urządzenia ze strumienia wejściowego.

Obiekt klasy *DataOutputStream* jest używany do zapisu wartości typów danych podstawowych do strumienia wyjściowego. Klasa *DataInputStream* zawiera metody *readXxx()* umożliwiające odczytywanie wartości danych typu *Xxx*. Przykładowo do odczytu wartości *int* dostępna jest metoda *readInt()*. Klasa wspiera również wczytywanie łańcuchów za pomocą metody *readUTF()*.

Klasa *DataOutputStream* zawiera metodę *writeXxx(Xxx value)*, odpowiadającą każdej metodzie *readXxx()* klasy *DataInputStream*. Klasa obsługuje nagrywanie ciąg znaków do strumienia wyjściowego za pomocą metody *writeUTF (String tekst)*. Należy zauważyć, że metody zapewniają wygodny sposób odczytywania i zapisywania wartości typów danych i ciągów znaków przy użyciu danych wejściowych i wyjściowych strumieni.

Przykładowy program zapisujący wartości typów danych do pliku o rozszerzeniu *.dat* został pokazany poniżej.

```
String destFile = "dane4.dat";  
try (DataOutputStream dos = new DataOutputStream  
    (new FileOutputStream(destFile))) {  
    dos.writeInt(765);  
    dos.writeDouble(6789.50);  
    dos.writeBoolean(true);  
    dos.writeUTF("Java Input/Output format .dat!");  
    dos.flush();  
    System.out.println("Data has been written to "+  
        (new File(destFile)).getAbsolutePath());  
}  
catch (FileNotFoundException e) {  
    System.out.println("Błąd!" +  
        e.getLocalizedMessage());  
catch (IOException e) {e.printStackTrace();}
```

```
String srcFile = "dane4.dat";  
try (DataInputStream dis = new DataInputStream  
    ( new FileInputStream(srcFile))) {  
    int intValue = dis.readInt();  
    double doubleValue = dis.readDouble();  
    boolean booleanValue = dis.readBoolean();  
    String msg = dis.readUTF();  
    System.out.println(intValue);  
    System.out.println(doubleValue);  
    System.out.println(booleanValue);  
    System.out.println(msg); }  
catch (FileNotFoundException e) {  
    System.out.println("Bład!" +  
        e.getLocalizedMessage()); }  
catch (IOException e) {e.printStackTrace();}
```

Otrzymane wyniki





```
run:  
Data has been written to D:\Java Ora\C6\Beans\C6d1\dane4.dat  
BUILD SUCCESSFUL (total time: 0 seconds)
```






```
run:  
765  
6789.5  
true  
Java Input/Output format .dat!  
BUILD SUCCESSFUL (total time: 0 seconds)
```

Serializacja obiektów

Proces konwersji obiektu w pamięci do sekwencji bajtów i przechowywania sekwencji bajtów na nośniku pamięci masowej, takim jak plik jest nazywany serializacją obiektu. Otrzymane sekwencje bajtów w wyniku serializacji możemy przechowywać w plikach lub bazach danych lub przesyłać przez sieć. Proces odczytywania sekwencji bajtów wytwarzany w procesie serializacji i przywracanie obiektu do pamięci jest nazywany deserializacją obiektu. Serializację można traktować jako zapisywanie obiektu z pamięci na nośniku pamięci masowej i deserializacja jako czytanie obiektu do pamięci z nośnika pamięci masowej. Przedstawiony przykład ilustruje proces serializacji i deserializacji obiektów.

```
package d6e1;
import java.io.Serializable;
public class Osoba implements Serializable{
    private String nazwisko    = "Nieznane";
    private String plec = "Nieznana" ;
    private double wzrost = Double.NaN;
    public Osoba(String nazwisko,
        String plec, double wzrost) {
        this.nazwisko    = nazwisko;
        this.plec = plec;
        this.wzrost = wzrost; }
    public void setNazwisko(String nazwisko){
        this.nazwisko = nazwisko; }
}
```

```
    public String getNazwisko() {  
        return nazwisko;    }  
    public void setWzrost(double wzrost) {  
        this.wzrost = wzrost;    }  
    public double getWzrost() {  
        return wzrost;    }  
    @Override  
    public String toString() {  
        return "Nazwisko: " + this.nazwisko +  
            ", Płeć: " + this.plec +  
            ", Wzrost: " + this.wzrost ;    }  
}
```

```
package d6e1;
import java.io.*;
public class D6e1 {
    public static void serialize(String fN) {
        Osoba jan = new Osoba("Jan", "Męska", 1.8);
        // Utworzenie obiektu
        File f0 = new File(fN);
        try (ObjectOutputStream oos = new
            ObjectOutputStream(new FileOutputStream(f0))) {
            System.out.println("Obiekt zapisany do " +
                f0.getAbsolutePath());
        }
    }
}
```

```
oos.writeObject(jan);  
System.out.println(jan);  
// Wyświetlenie zapisu  
jan.setNazwisko("Jan Woźnica");  
jan.setWzrost(1.75);  
oos.writeObject(jan);  
System.out.println(jan);  
// Wyświetlenie co nadpisano  
}  
catch (IOException e1) {  
    e1.printStackTrace();  
}  
}
```

```
public static void deserialize(String fN) {  
    File f0 = new File(fN);  
    try (ObjectInputStream ois =new  
        ObjectInputStream(new FileInputStream(f0))) {  
        Osoba jan1 = (Osoba)ois.readObject();  
        Osoba jan2 = (Osoba)ois.readObject();  
        System.out.println("Obiekt przeczytany z " +  
            f0.getAbsolutePath());  
        System.out.println(jan1);  
        System.out.println(jan2);  
    }  
    catch (IOException | ClassNotFoundException e) {  
        e.printStackTrace();    }  
}
```

```
public static void main(String[] args) {  
    String fN = "osoby.ser";  
    // Zapisanie obiektu do strumienia  
        serialize(fN);  
    System.out.println  
    ("-----");  
    // Przeczytanie obiektu z powrotem  
        deserialize(fN);  
    }  
}
```

Serializacja obiektów

```
run:
```

```
Obiekt zapisany do D:\Java Ora\C6\Beans\D6e1\osoby.ser
```

```
Nazwisko: Jan, Płeć: Męska, Wzrost: 1.8
```

```
Nazwisko: Jan Woźnica, Płeć: Męska, Wzrost: 1.75
```

```
-----
```

```
Obiekt przeczytany z D:\Java Ora\C6\Beans\D6e1\osoby.ser
```

```
Nazwisko: Jan, Płeć: Męska, Wzrost: 1.8
```

```
Nazwisko: Jan, Płeć: Męska, Wzrost: 1.8
```

Czytanie i zapis łańcuchów znaków

Większość klasy obsługujących strumienie wejścia wyjścia jest oparta czytaniu i zapisie bajtów. Na przykład odczyt i zapis danych pierwotnych i obiektów są zawsze oparte na bajtach. W związku z tym nie masz żadnych klas w rodzinie klasy *io* odpowiadające strumieniom danych zapisanych w różnym kodowaniu znaków. . Dwie klasy, *InputStreamReader* i *OutputStreamWriter*, w rodzinie klasy *Reader* i *Writer* są pomostem pomiędzy klasami opartymi na znak i strumieniami bajtów. Przykładowo, jeśli skonstruujemy obiekt klasy *InputStream* o nazwie *iso* będziemy mogli utworzyć obiekt instancji *Reader* :

```
Reader reader = new InputStreamReader(iso, "UTF-8");  
// Utworzenie obiektu Reader strumienia wejściowego  
// używającego kodowania UTF-8
```

Zadanie. Należy napisać program wczytujący linie tekstu z konsoli użytkownika i zapisujący kolejno przeczytane linie tekstu do pliku dane.txt w systemie kodowani UTF-8.

```
package c6f;
import java.io.*;
import java.util.*;
public class C6f {
    public static void main(String[] args)
        throws FileNotFoundException {
        Scanner in = new Scanner(System.in, "Cp1250");
        String destFile = "dane.txt";
        // Nazwa pliku
        System.out.println
            ("Wprowadź kolejne linie tekstu:");
```

```
try{
    FileOutputStream oso =
        new FileOutputStream(destFile);
// Writer writer =
// new OutputStreamWriter(oso, "UTF8");
// new OutputStreamWriter(oso, "windows-1250");
// new OutputStreamWriter(oso, "Cp1250");
    Writer wr = new BufferedWriter
        ( new OutputStreamWriter(oso, "UTF-8"));
    String s;
    while(!(s = in.nextLine()).equals("quit"))
        wr.append(s).append("\r\n");
}
```

```
        wr.flush();  
// Opróżnienie bufora i zapisanie pliku  
        System.out.println("Text was written to " +  
            (new File(destFile)).getAbsolutePath());  
    }  
    catch(IOException e) {  
        System.out.println  
            ("Błąd!" + e.getMessage());  
    }  
    catch(Exception e) {  
        System.out.println  
            ("Błąd!" + e.getMessage());  
    }  
}  
}
```

Sprawdzenie działanie programu

run:

Wprowadź kolejne linie tekstu:

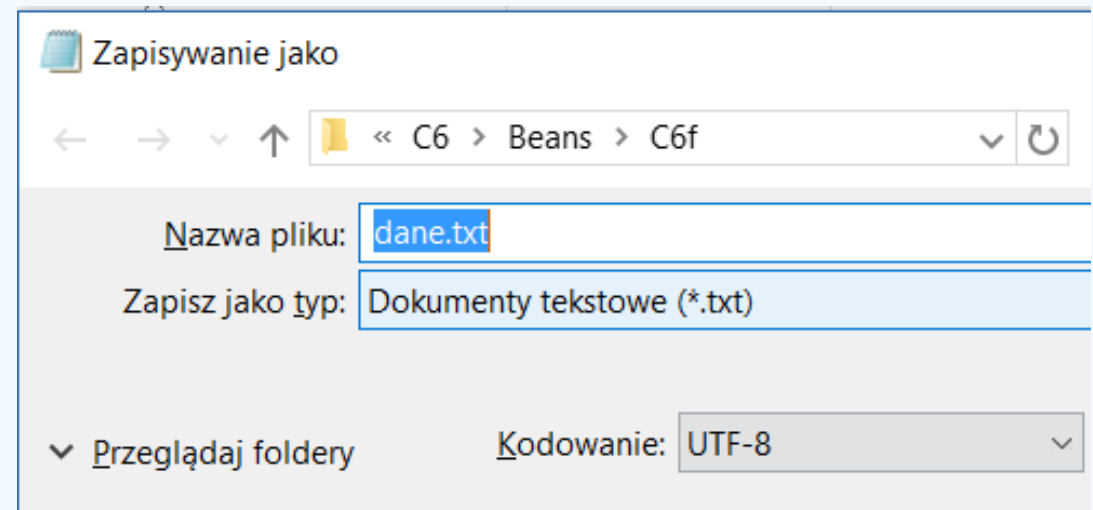
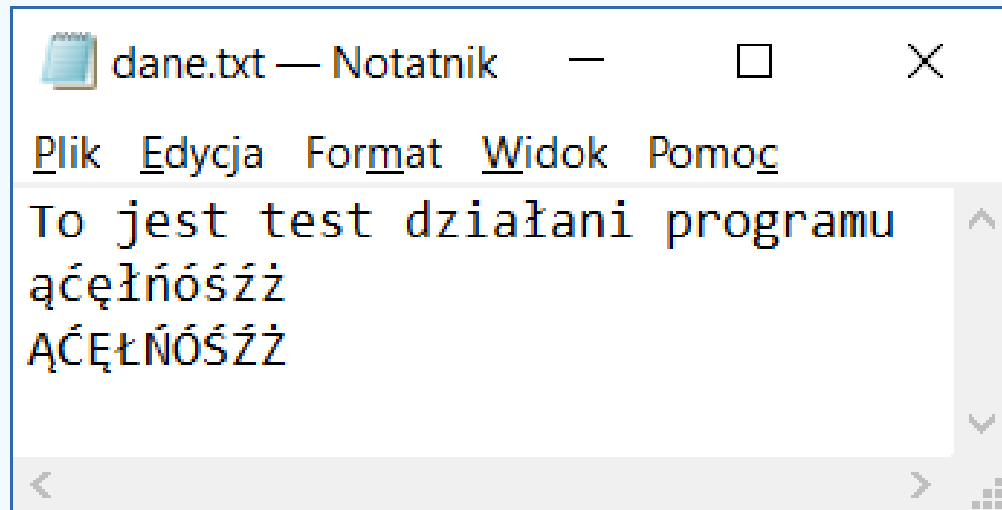
To jest test działania programu

ąćęłńóśźź

ĄĆĘŁŃÓŚŹŹ

quit

Text was written to D:\Java Ora\C6\Beans\C6f\dane.txt

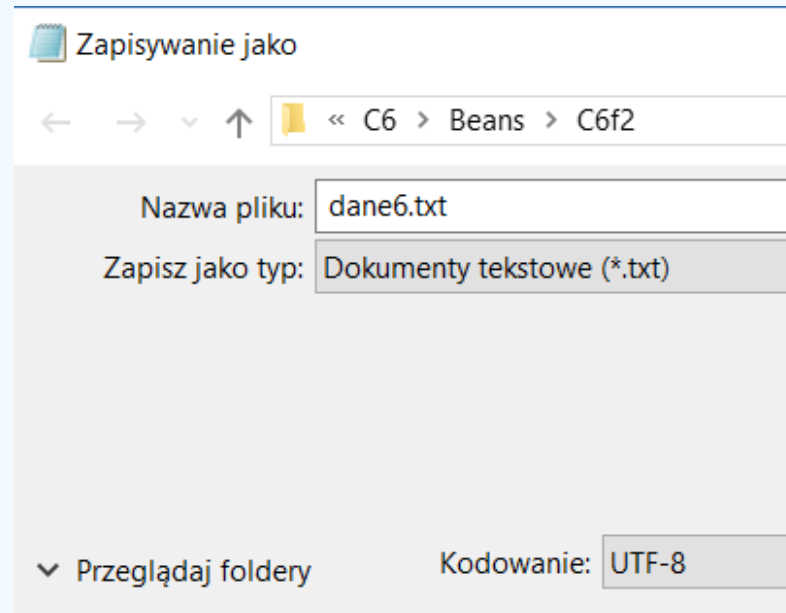
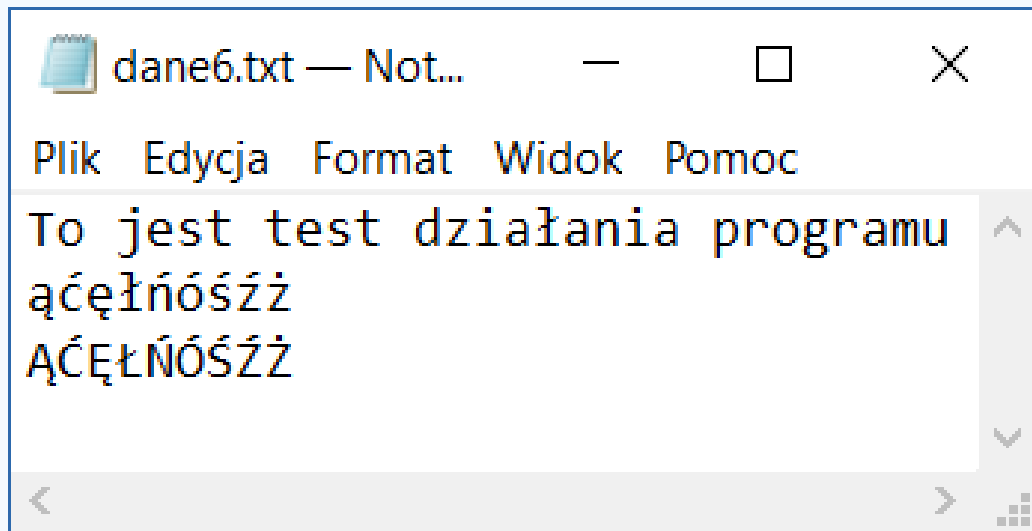


Zadanie. Należy napisać program wypisujący zawrtość pliku w kodowaniu UTF-8 linia po linii w konsoli użytkownika.

```
package c6f2;
import java.io.*;
import java.nio.*;
public class C6f2 {
    public static void main(String[] args) {
        File dF = new File("dane6.txt");
        try{
            FileInputStream iso = new FileInputStream(dF);
            Reader rd = new BufferedReader
                (new InputStreamReader(iso, "UTF-8"));
            CharBuffer chbuff = CharBuffer.allocate(1024);
```

```
while (rd.read(chbuff) > 0) {  
    chbuff.flip();  
    String s = chbuff.toString();  
    System.out.print(s);  
    chbuff.clear();  
}  
rd.close();  
}  
catch (IOException e) {System.out.println("Błąd!" +  
    e.getMessage());}  
catch (Exception e) {e.printStackTrace();}  
}  
}
```

Sprawdzenie działanie programu



run:

```
To jest test działania programu  
ąćęłńóśźź  
ĄĆĘŁŃÓŚŹŹ
```

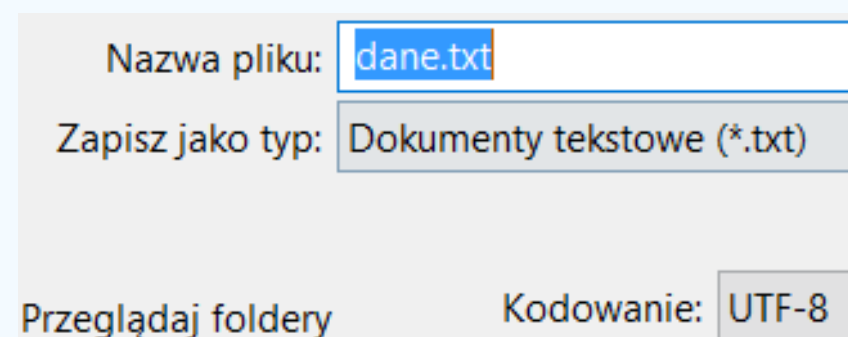
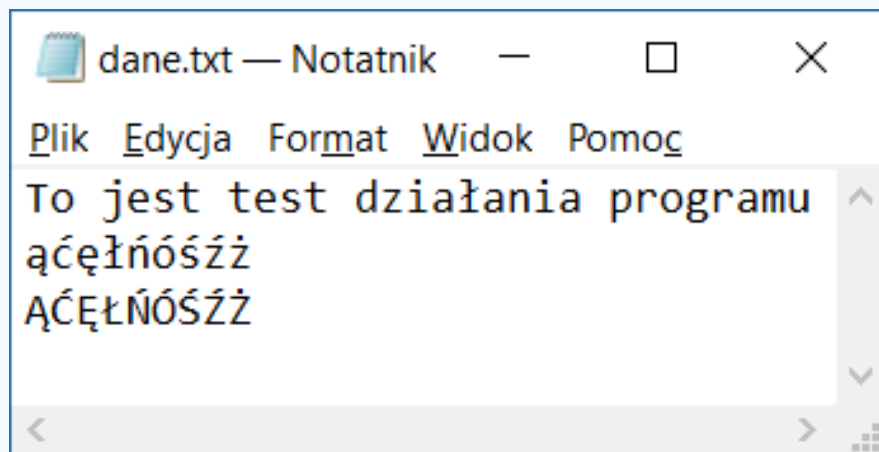
Zapisywanie pliku w kodowaniu UTF-8

```
String dF = "dane.txt";  
try (BufferedWriter bw =  
    new BufferedWriter(new FileWriter(dF))) {  
    // Write the text to the writer  
    bw.append("To jest test działania programu");  
    bw.newLine();  
    bw.append("ąćęłńóśźż");  
    bw.newLine();  
    bw.append("ĄĆĘŁŃÓŚŹŻ");  
    bw.newLine();  
    bw.flush();  
    System.out.println("Text was written to " +  
        (new File(dF)).getAbsolutePath());  
}
```

```
catch (FileNotFoundException e) {  
    System.out.println("Błąd!" +  
        e.getLocalizedMessage());  
}  
catch (IOException e) {  
    e.printStackTrace();  
}
```

run:

Text was written to D:\Java Ora\C6\Beans\C6f3\dane.txt



Odczytywanie pliku w kodowaniu UTF-8

```
String sF = "dane.txt";  
String text = null;  
BufferedReader br =  
    new BufferedReader(new FileReader(sF));  
while ((text = br.readLine()) != null)  
    System.out.println(text);  
br.close();
```

run:

To jest test działania programu
ąćęłńóśźź
ĄĆĘŁŃÓŚŹŹ

Nazwa pliku:	dane.txt
Zapisz jako typ:	Dokumenty tekstowe (*.txt)
Ukryj foldery	Kodowanie: UTF-8

Czytanie całej zawartości plików

Pakiet NIO.2.API udostępnia metody umożliwiające czytanie całej zawartości plików:

- **static byte[]** readAllBytes(**Path** *path*)
- **static List<String>** readAllLines(**Path** *path*)
- **static List<String>** readAllLines(**Path** *path*, **Charset** *cs*)

Wszystkie metody mogą wyrzucać wyjątek *IOException*. Metoda *readAllBytes()* czyta wszystkie bajty pliku. Metoda *readAllLines()* czyta zawartość całego pliku jako linii tekstu. Wszystkie wersje, które mają na wejściu tylko argument *Path* pliku źródłowego zakładają, że zawartość pliku została zapisana w kodowaniu UTF-8.

UWAGA: Wszystkie metody są przeznaczone do czytania małych plików danych. Metody automatycznie podejmują akcję otwierania i zamykania plików.

```
import java.nio.file.*;
import java.util.List;
import java.nio.charset.Charset;
import java.io.IOException;
public class C6a {
    public static void main(String[] args) {
        Charset cs = Charset.forName("Cp1250");
        Path source = Paths.get("dane6.txt");
        try {
            List<String> lines =
                Files.readAllLines(source, cs);
            // Przeczytanie wszystkich linii pliku
            for (String line : lines)
                System.out.println(line);
            // Wypisanie wszystkich linii tekstu
        }
    }
}
```

```
catch (NoSuchFileException e) {  
    System.out.println(source.toAbsolutePath()  
        + " nie istnieje"); }  
catch (IOException e) {  
    e.printStackTrace(); }  
}  
}
```

run:

To jest test działania
ąćęłńóśźź
ĄĆĘŁŃÓŚŹŹ

Zapisywanie zawartości plików

Pakiet NIO.2.API udostępnia metody umożliwiające zapisywanie do plików:

- **static Path** write(**Path** *path*, **byte[]** *bytes*, **OpenOption...** *options*)
- **static Path** write(**Path** *path*, **Iterable**<? extends *CharSequence*> *lines*, **OpenOption...** *options*)
- **static Path** write(**Path** *path*, **Iterable**<? extends *CharSequence*> *lines*, **Charset** *cs*, **OpenOption...** *options*)

UWAGA: Wszystkie metody są przeznaczone do czytania małych plików danych. Metody automatycznie podejmują akcję otwierania i zamykania plików.

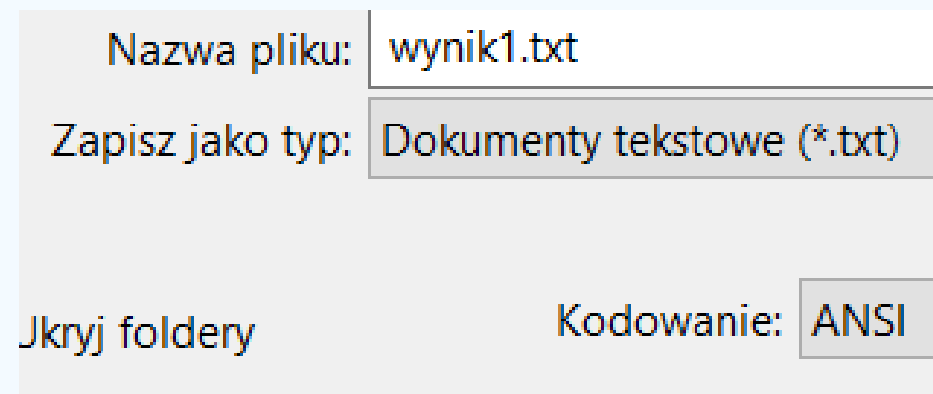
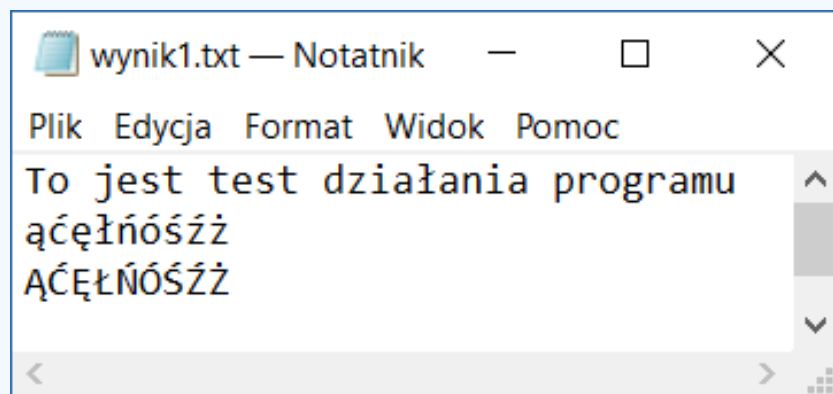
```
import java.io.IOException;
import java.nio.charset.Charset;
import java.nio.file.*;
import java.util.*;
import static
    java.nio.file.StandardOpenOption.*;
public class C6a1 {
    public static void main(String[] args){
        List<String> texts = new ArrayList<>();
        texts.add
            ("To jest test działania programu");
        texts.add("ąćęłńóśźż");
        texts.add("ĄĆĘŁŃÓŚŹŻ");
    }
}
```

```
Path dt = Paths.get("wynik1.txt");
Charset cs = Charset.forName("Cp1250");
try{
    Path p = Files.write(dt, texts,
        cs, TRUNCATE_EXISTING, CREATE);
    System.out.println("Tekst zapisany do: " +
        p.toAbsolutePath());
}
catch (IOException e) {
    e.printStackTrace();
}
}
```

Sprawdzenie działanie programu

```
run:
```

```
Tekst zapisany do: D:\Java Ora\C6\Beans\C6a1\wynik1.txt
```



Zadanie

W pliku we.txt w kodowaniu UTF-8 zapisane są linie tekstu. Należy przeczytać linie tekstu do programu, następnie posortować przeczytane łańcuch wg. polskiej kultury językowej i zapisać do pliku wy.txt w kodowaniu UTF-8.

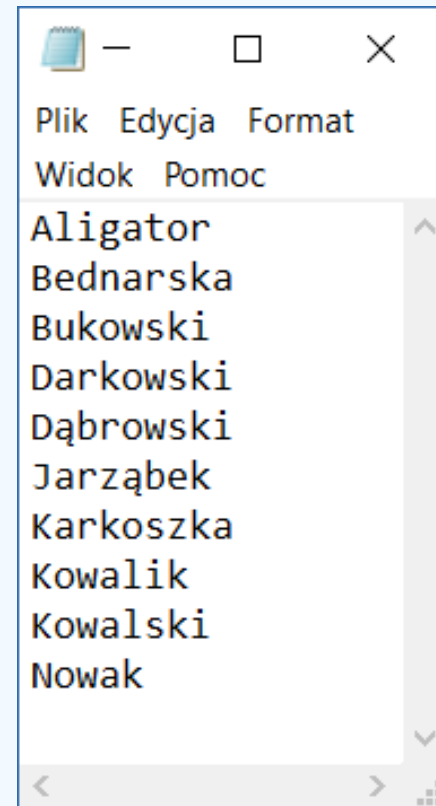
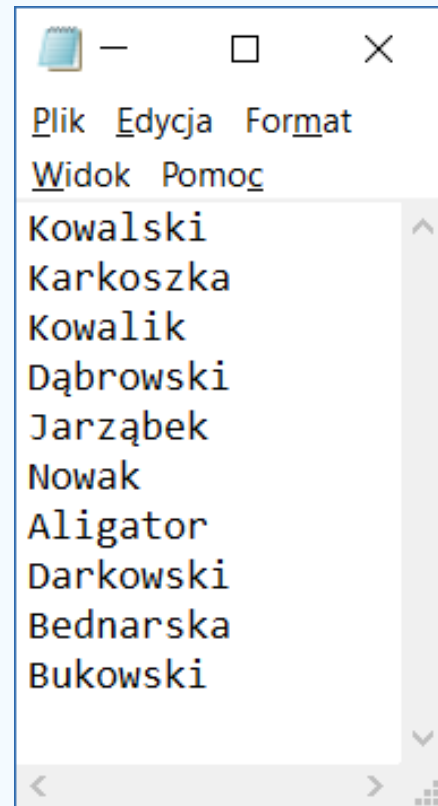
```
Path se = Paths.get("we.txt");  
Path dt = Paths.get("wy.txt");
```

```
try {
    List<String> lines =
        Files.readAllLines(se);
// Przeczytanie wszystkich linii pliku
    Locale lp = new Locale("pl", "PL");
    Collator col = Collator.getInstance(lp);
    Collections.sort(lines, col);
    Path p = Files.write(dt, lines);
    System.out.println("Tekst zapisany do: " +
                        p.toAbsolutePath());
}
catch (Exception e) {
    System.out.println("Błąd!" +
                        e.getLocalizedMessage());
}
```

Sprawdzenie działanie programu

```
run:
```

```
Tekst zapisany do: D:\Java Ora\C6\Beans\C6g\wy.txt
```



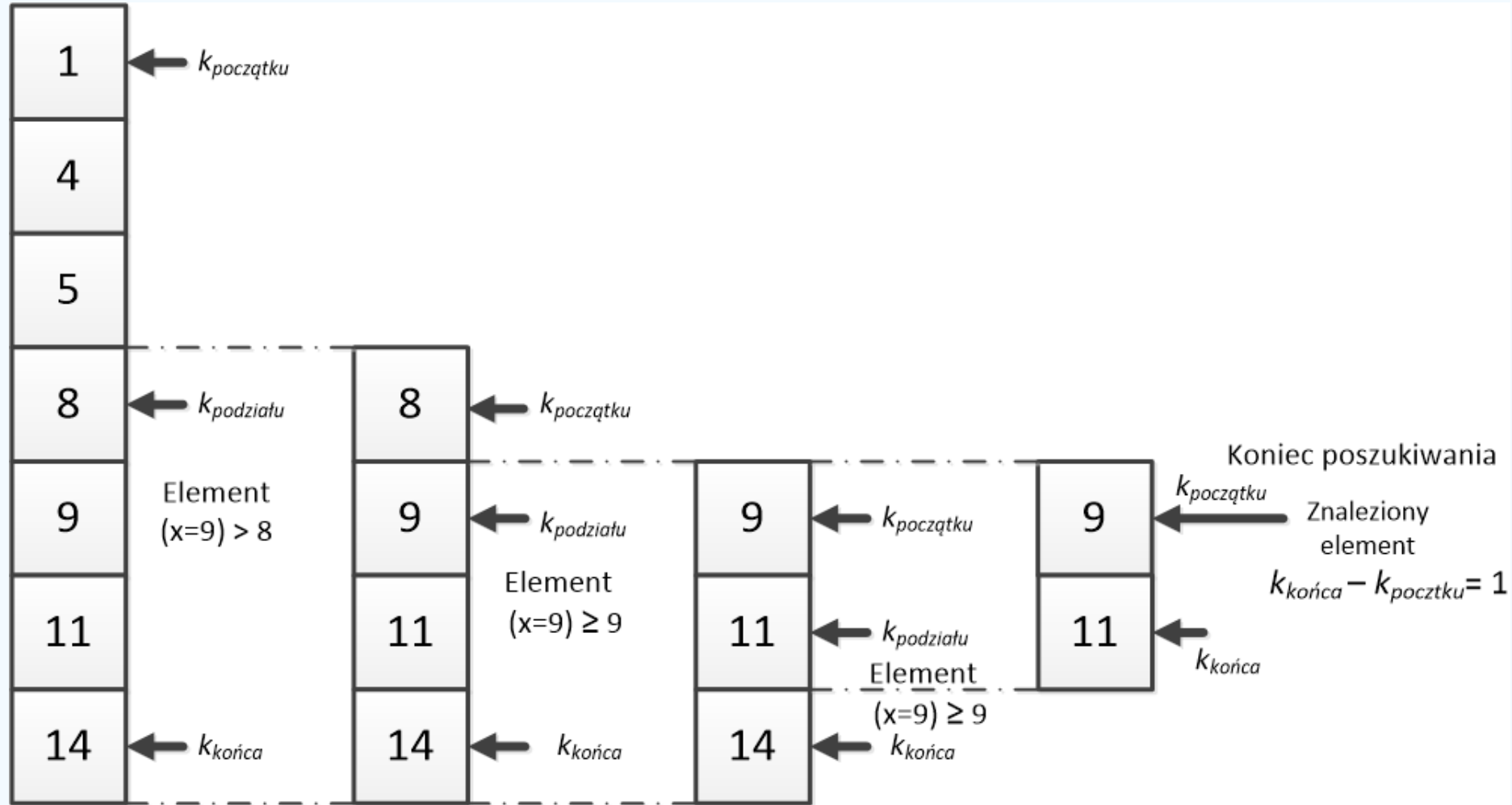
Wyszukiwanie informacji w bazach NoSQL

Efektywnym rozwiązaniem w wyszukiwaniu wzorców w dużych bazach danych jest podzielenie zbioru przeszukiwanych rekordów na mniejsze części. Przedstawiony poniżej algorytm dzieli uporządkowany zbiór rekordów na dwie części. Poszukiwanie następuje w każdym kolejnym kroku w dwukrotnie mniejszym zbiorze danych. Podział uporządkowanej tablicy rekordów następuje zgodnie z wyborem punktu podziału:

$$k_{podziału} = \left\lfloor \frac{k_{początku} + k_{końca}}{2} \right\rfloor$$

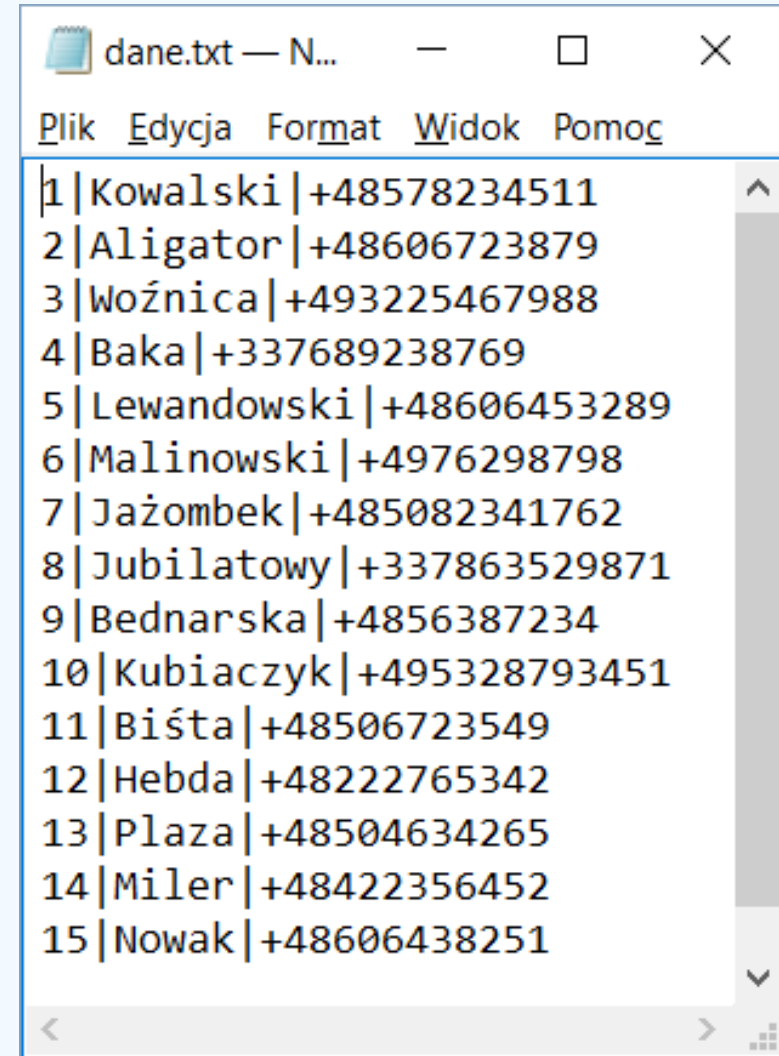
Gdzie $k_{początku}$ – oznacza indeks od którego rozpoczynamy poszukiwanie, $k_{końca}$ – oznacza indeks na którym kończymy wyszukiwanie i $k_{podziału}$ – oznacza indeks według którego następuje podział. Nie ma potrzeby rozpatrywania parzystości elementów w zbiorze danych. Binarny podział zawęża przeszukiwany zbiór danych dwukrotnie wykorzystując informację uporządkowania rekordów w przeszukiwanej tablicy. Poniższy rysunek pokazuje w jaki sposób działa algorytm wyszukiwania binarnego na liczbach całkowitych.

Metoda wyszukiwania binarnego



Program wyszukiwania pracownika w bazie NoSQL

Przedstawiony program wczytuje rekordy z bazy danych zapisane wierszami. W każdym wierszu jest zapisany rekord mający trzy pola oddzielone pionową kreską (|). Pierwsze pole zawiera identyfikator, w drugim polu zapisane jest nazwisko, a w trzecim telefon. Plik z zapisanymi rekordami może wyglądać jak na rysunku.



The screenshot shows a text editor window titled 'dane.txt — N...'. The menu bar includes 'Plik', 'Edycja', 'Format', 'Widok', and 'Pomoc'. The text area contains 15 lines of data, each representing an employee record. Each record is formatted as 'ID|Name|Phone', where the ID is a number from 1 to 15, the Name is a Polish surname, and the Phone is a 12-digit number starting with a plus sign. A vertical scrollbar is visible on the right side of the text area.

ID	Name	Phone
1	Kowalski	+48578234511
2	Aligator	+48606723879
3	Woźnica	+493225467988
4	Baka	+337689238769
5	Lewandowski	+48606453289
6	Malinowski	+4976298798
7	Jażombek	+485082341762
8	Jubilatowy	+337863529871
9	Bednarska	+4856387234
10	Kubiaczyk	+495328793451
11	Biśta	+48506723549
12	Hebda	+48222765342
13	Plaza	+48504634265
14	Miler	+48422356452
15	Nowak	+48606438251

```
package c6h;
import java.text.*;
import java.util.*;
public class Pracownik implements Comparable{
    private int id;
    private String nazwisko, telefon;
// Pola klasy.
    public Pracownik() { }
// Konstruktor domyślny.
    public Pracownik(int id, String nazwisko,
                      String telefon){
// Przeciążenie konstruktora.
        this.id = id;
        this.nazwisko = nazwisko;
        this.telefon = telefon;
    }
}
```

```
    @Override
    public String toString() {
// Przeciążenie metody ToString().
        return String.format
            ("%3d %-12s %-10s", id, nazwisko,
                                   telefon);
    }
    private static int compareIgnoreCase
        (String a1, String a2)
    {
        Locale locale =new Locale("pl","PL");
// Utworzenie obiektu lokalnych ustawień
```

```
        Collator col =  
            Collator.getInstance(locale);  
//Utworzenie komparatora instancji polskiej.  
        col.setStrength(Collator.PRIMARY);  
// Wpisanie sposobu porównywania znaków.  
        return col.compare  
            (a1.toUpperCase(locale),  
             a2.toUpperCase(locale));  
// Zwrócenie wyniku porównania.  
    }
```

```
@Override
public int compareTo(Object obj) {
    if (this==obj) return 0;
    if (this == null) return -1;
    if (this.getClass() != obj.getClass()
        ) return 1;
    Pracownik e = (Pracownik) obj ;
    int t = compareIgnoreCase
              (this.nazwisko,e.nazwisko);
    return t;
}
}
```

```
package c6h;
public class Rekord {
    public void szukaj_b(Pracownik[] a, String s) {
        // Metoda wyszukiwania binarnego.
        Pracownik x =new Pracownik(0,s,"");
        // Utworzenie obiektu klasy Dane.
        int lp;
        // Deklaracja indeksu podzału.
        int n = a.length;
        // Pobranie ilości rekordów.
        int ld =0;
        // Indeks początku przeszukiwania.
        int lg=n-1;
        // Indeks końca przeszukiwania.
```

```
    while ((lg-ld)>1) {  
// Wykonywanie zacieśniania przedziału poszukiwania.  
    lp=(ld+lg)/2;  
// Obliczony indeksu podziału  
    if(x.compareTo(a[lp])<0) lg=lp;  
// Wyznaczenie indeksów nowego przedziału poszukiwania  
// w zależności od dokonanego porównania.  
    else ld=lp;  
    }  
    if(x.compareTo(a[ld])==0) {  
// Sprawdzenie, czy została znaleziona osoba.  
        System.out.println(a[ld]);  
        return;}  
}
```

```
        if(x.compareTo(a[lg])==0) {  
// Sprawdzenie, czy została znaleziona osoba.  
            System.out.println(a[lg]);  
            return;}  
        System.out.println  
            ("Nie znalazłem osoby w bazie.");  
    }  
}  
package c6h;  
import java.io.*;  
import java.util.*;  
public class C6h {  
    public static void main(String[] args) {
```

```
String sF = "dane3.txt";
Scanner in =
    new Scanner(System.in, "Cp1250");
try{
    BufferedReader sr1 = new
        BufferedReader(new FileReader(sF));
    int n = 0;
    String s;
    while( (sr1.readLine()) != null)
// Określenie ilości rekordów w pliku.
        n++;
    n--;
    sr1.close();
}
```

```
Pracownik [] a = new Pracownik[n];
System.out.println("n="+n);
sr1 = new BufferedReader
        (new FileReader(sF));
sr1.readLine();
for(int i=0;i<n;i++){
    s=sr1.readLine();
    String [] st = s.split("[]");
// Podział rekordu na pola.
    a[i] = new Pracownik
        (Integer.parseInt(st[0]),st[1],st[2]);
// Dopisanie rekordu do uchwytu.
}
sr1.close();
```

```
Arrays.sort(a);  
for (Pracownik b: a)  
    System.out.println(b);  
System.out.print  
    ("Podaj nazwisko poszukiwanej osoby: ");  
String s3 = in.nextLine();  
Rekord re=new Rekord();  
// Konstrukcja obiektu klasy rekord.  
    re.szukaj_b(a,s3);  
// Wyszukanie nazwiska w bazie.  
    }  
    catch (Exception e) {  
        System.out.println  
            ("Bład!" + e.getLocalizedMessage());  
    }  
}
```

Sprawdzenie działanie programu

run:

n=15

2	Aligator	+48606723879
4	Baka	+337689238769
9	Bednarska	+4856387234
11	Biśta	+48506723549
12	Hebda	+48222765342
7	Jażombek	+485082341762
8	Jubilatowy	+337863529871
1	Kowalski	+48578234511
10	Kubiaczyk	+495328793451
5	Lewandowski	+48606453289
6	Malinowski	+4976298798
14	Miler	+48422356452
15	Nowak	+48606438251
13	Plaza	+48504634265
3	Woźnica	+493225467988

Podaj nazwisko poszukiwanej osoby: woźnica

3 Woźnica +493225467988

run:

n=15

2	Aligator	+48606723879
4	Baka	+337689238769
9	Bednarska	+4856387234
11	Biśta	+48506723549
12	Hebda	+48222765342
7	Jażombek	+485082341762
8	Jubilatowy	+337863529871
1	Kowalski	+48578234511
10	Kubiaczyk	+495328793451
5	Lewandowski	+48606453289
6	Malinowski	+4976298798
14	Miler	+48422356452
15	Nowak	+48606438251
13	Plaza	+48504634265
3	Woźnica	+493225467988

Podaj nazwisko poszukiwanej osoby: Alen

Nie znalazłem osoby w bazie.